Pós-Graduação em Machine Learning,
Deep Learning e Inteligência Artificial

Operacionalização de Modelos com MLOps [26E1_2]

Prof. Ícaro Augusto Maccari Zelioli

Avaliação de Risco de Crédito

12 de abril de 2026

Projeto: Operacionalização de Modelos com MLflow e Streamlit

Aluno:

- Fabio Ferreira Figueiredo

Link GitHub:

https://github.com/fabioffigueiredo/pd_operacionaliza-o_modelos-_mlops

Descrição do Problema, MLOps e Pipeline

Contexto do Problema:

O dataset **Give Me Some Credit** contém informações de aproximadamente 150.000 tomadores de crédito. O objetivo é prever inadimplência severa nos próximos 2 anos, usando a variável alvo binária `SeriousDlqin2yrs`.

Nesta segunda etapa, o foco deixa de ser apenas comparação de modelos e passa a ser **operacionalização**: organização de código, rastreabilidade com MLflow, seleção controlada de modelo campeão e inferência reproduzível via Streamlit.

O contexto regulatório continua central. Em crédito, a solução precisa equilibrar desempenho, interpretabilidade e governança, considerando exigências associadas à LGPD e à transparência esperada em decisões automatizadas.

Guia de Navegação — Rubrica de Avaliação

> **Nota ao avaliador:** a estrutura do relatório foi reorganizada para seguir diretamente a ordem da rubrica do professor. Cada competência aparece como uma seção principal, facilitando a validação do que foi demonstrado em cada critério.

Fase da Rubrica	Competência Avaliada	Seção do Relatório
Competência 1	Estruturação do projeto, contexto, planejamento e transição do notebook para entrega	Seção 1
Competência 2	Pipelines de dados, qualidade, features e análise de redução de dimensionalidade	Seção 2
Competência 3	Experimentos reprodutíveis, comparação de abordagens e rastreamento com MLflow	Seção 3
Competência 4	Operacionalização, inferência, métricas de negócio, monitoramento, drift e re-treino	Seção 4
Fechamento	Conclusões e próximos passos	Seção 5

1. Competência 1 — Estruturação do Projeto, Contexto e Entrega

1.1 Introdução e Mudança de Paradigma

O PD1 focou na experimentação isolada de modelos dentro de um Jupyter Notebook, comparando algoritmos (Perceptron, Árvore de Decisão, Random Forest) para o problema de credit scoring. O PD2 representa uma mudança fundamental de perspectiva: do cientista de dados que experimenta para o **engenheiro de ML que opera sistemas**.

O foco deixa de ser "qual modelo tem o melhor F1?" e passa a ser: como construir um sistema de ML **reprodutível, rastreável e operacionalizável** em produção?

Esta transição exigiu a reestruturação completa do código em módulos Python independentes, a integração com MLflow para rastreamento de experimentos, a comparação sistemática de técnicas de redução de dimensionalidade, e o deploy do modelo via Streamlit.

Objetivo técnico: construir um pipeline reproduzível para previsão de `SeriousDlqin2yrs`, com separação clara entre preparo de dados, treinamento, rastreamento experimental e inferência.

Métricas de negócio associadas: reduzir o risco de **falsos negativos** (aprovar quem tende a inadimplir), sem ignorar o custo de **falsos positivos** (recusar bons pagadores), respeitando o contexto regulatório de crédito.

1.2 Reestruturação do Código: Do Notebook ao Sistema Modular

1.2.1 Problemas do Notebook

O notebook `projeto_credito_supervisionado.ipynb` (2.094 linhas, 498 KB) apresentava problemas típicos para operacionalização:

- **Estado global implícito:** variáveis de uma célula afetavam silenciosamente as seguintes
- **Não-reprodutibilidade:** ordem de execução das células importava; células podiam ser executadas fora de ordem
- **Impossibilidade de teste unitário:** lógica de negócio misturada com visualizações e narrativa
- **Sem versionamento de lógica:** mudanças no pré-processamento ou hiperparâmetros não eram rastreadas
- **Impossibilidade de integração:** nenhum script externo podia importar funções do notebook

1.2.2 Arquitetura Modular Adotada

```
src/
├── data_processing.py    # Separação de concerns: ingestão e pré-processamento
└── train.py              # Orquestração do treinamento com MLflow
config/
└── pipeline.yaml         # Configuração centralizada (fonte única de verdade)
app/
└── app.py                # Camada de apresentação desacoplada da lógica de ML
```

Princípio central adotado: cada módulo tem uma única responsabilidade e pode ser testado independentemente. O `data_processing.py` pode ser importado tanto pelo `train.py` quanto por um script de inferência batch, sem copiar código.

1.2.3 Configuração Centralizada

O arquivo `config/pipeline.yaml` elimina os "magic numbers" espalhados pelo código. Hiperparâmetros, paths, e configurações de validação cruzada estão em um único lugar. Isso permite: - Reproduzir experimentos com configurações diferentes sem editar código - Auditar facilmente quais parâmetros foram usados em cada experimento (via MLflow) - Facilitar o processo de CI/CD: pipelines automatizados leem o YAML

---

2. Competência 2 — Pipelines de Dados, Features e Redução de Dimensionalidade

2.1 Qualidade de Dados e Tratamento

2.1.1 Diagnóstico de Problemas

A análise exploratória do PD1 identificou dois tipos principais de problemas:

Feature	Problema	Proporção afetada
`MonthlyIncome`	Valores ausentes (NaN)	~20% dos registros
`NumberOfDependents`	Valores ausentes (NaN)	~2,5% dos registros

Feature	Problema	Proporção afetada
`DebtRatio`	Outliers absurdos (> 1000)	< 1% dos registros
`RevolvingUtilizationOfUnsecuredLines`	Outliers (> 1.0 impossível fisicamente)	< 1% dos registros

2.1.2 Estratégia de Tratamento: Outliers por Capping

Decisão técnica: Capping no percentil 99 para `DebtRatio` e `RevolvingUtilizationOfUnsecuredLines`.

Justificativa: - `DebtRatio` representa a razão entre pagamentos de dívida e renda. Valores acima de 1.000 são matematicamente impossíveis em contextos reais (indicam erros de entrada de dados). - O capping preserva a distribuição da população real enquanto elimina o efeito distorcivo desses outliers no `StandardScaler` — sem capping, a padronização comprimiria toda a distribuição real em um intervalo minúsculo. - A alternativa (remoção dos registros) eliminaria ~1.500 linhas potencialmente válidas onde apenas uma feature apresenta problema.

Limitação documentada: O percentil 99 foi calculado no dataset completo antes do `train_test_split`. Em produção, o threshold deveria ser calculado exclusivamente no conjunto de treino e armazenado como parâmetro do pipeline. Esta simplificação é aceitável no contexto acadêmico e está documentada no código.

2.1.3 Estratégia de Tratamento: Valores Ausentes

Decisão técnica: `SimpleImputer(strategy='median')` dentro do sklearn Pipeline.

Justificativa: - A mediana é mais robusta que a média na presença de outliers residuais (após o capping, ainda pode haver valores extremos legítimos em `MonthlyIncome`). - Imputação dentro do Pipeline garante que a mediana é calculada APENAS no conjunto de treino e aplicada ao conjunto de teste — evitando data leakage. - A alternativa (remoção de linhas com NaN) eliminaria ~30.000 registros (~20% do dataset), reduzindo severamente a representatividade da amostra.

2.1.4 Impacto na Generalização

A presença de 20% de valores ausentes em `MonthlyIncome` representa um risco estrutural: se a ausência de renda estiver correlacionada com o comportamento de pagamento (hipótese plausível — clientes sem renda declarada podem ser mais propensos à inadimplência), a imputação pela mediana introduz um viés. Esta limitação é irresolvível pelos dados disponíveis e deve ser documentada para o time de negócio.

2.2 Redução de Dimensionalidade: Análise Comparativa

2.2.1 Motivação e Contexto

Com apenas 10 features e 150.000 registros, este dataset não sofre da maldição da dimensionalidade. A aplicação de redução de dimensionalidade serve aqui como experimento comparativo explícito, conforme exigido pela rubrica do PD2, não como necessidade técnica primária.

Foram testadas duas técnicas com características complementares: **PCA** (não supervisionada, baseada em variância) e **LDA** (supervisionada, baseada em separabilidade de classes).

2.2.2 PCA — Análise de Componentes Principais

Configuração: `PCA(n_components=0.95)` — sklearn seleciona automaticamente o número mínimo de componentes que explicam ≥ 95% da variância total.

Resultado observado: 9 componentes para atingir 95% da variância explicada. Isso confirma que, apesar de correlações entre as features de atraso, o espaço de features preserva informação relativamente distribuída entre múltiplas dimensões.

Impacto no F1-Score: Espera-se desempenho similar ao baseline ou ligeiramente inferior, pois a compressão de 10 para ~8 dimensões é modesta e o RF é robusto a correlações.

Impacto na interpretabilidade: Crítico. Os componentes principais são combinações lineares de todas as features originais. A feature `RevolvingUtilizationOfUnsecuredLines` — identificada no PD1 como o nó raiz da árvore de decisão mais importante — não existe mais como entidade identificável no espaço transformado. Isso é incompatível com os requisitos da **LGPD Art. 20** (direito à explicação de decisões automatizadas) e com as resoluções do **BACEN** que exigem transparência em modelos de crédito.

Conclusão sobre PCA: Tecnicamente viável como compressão, mas problemático do ponto de vista regulatório para um sistema de crédito no contexto brasileiro.

2.2.3 LDA — Análise Discriminante Linear

Configuração: `LDA(n_components=1)`.

Limitação matemática: Para classificação binária (2 classes), o máximo de componentes discriminantes é `min(n_classes - 1, n_features) = 1`. Esta não é uma escolha — é uma restrição algorítmica.

Impacto no F1-Score: Espera-se **queda significativa**. Projetar 10 dimensões em 1 único eixo, mesmo que esse eixo seja o que maximiza a separação entre classes, resulta em perda substancial de informação preditiva. Features como `MonthlyIncome` e `age` capturam aspectos do risco que são ortogonais entre si — colapsá-las em uma dimensão cria confusão.

Vantagem sobre PCA: O componente LDA tem interpretação: é a direção no espaço de features que melhor separa adimplentes de inadimplentes. Os coeficientes da transformação LDA indicam quais features mais contribuem para essa separação.

Conclusão sobre LDA: Inapropriado para este problema na forma univariada. Útil como análise exploratória (visualização 1D da separabilidade das classes), mas não como etapa de pré-processamento para produção.

2.2.4 Comparativo: Com vs. Sem Redução

Experimento	Redução	F1-Score (teste)	ROC-AUC	Interpretabilidade	Treino
RF + PCA	9 de 10 componentes (~95% var.)	**0.4354**	0.8555	Baixa (componentes latentes)	24.7 min
RF Baseline	Nenhuma	0.4308	0.8572	Alta (feature importances diretas)	19.4 min
RF + LDA	1 componente	0.3494	0.8156	Média (1 eixo interpretável)	14.9 min
DT Baseline	Nenhuma	0.3408	0.8544	Muito Alta (regras auditáveis)	45 seg

Análise da distribuição de variância (PCA): Com o capping adequado de outliers aplicado, as features passaram a ter distribuição de variância quase uniforme: PC1=21%, PC2=17%, PC3=13%, PC4=11%, PC5=8%, PC6=7%, PC7=7%, PC8=6%, PC9=6%, PC10=5%. São necessários **9 de 10 componentes** para atingir 95% da variância — evidência de que as features de crédito capturam aspectos ortogonais e independentes do comportamento financeiro.

---

3. Competência 3 — Experimentos Reprodutíveis com MLflow

3.1 Planejamento Experimental

Foram definidos quatro experimentos comparativos com a mesma base de treino/teste e a mesma métrica primária (`F1-Score`), para isolar o impacto da redução de dimensionalidade e comparar custo, desempenho e interpretabilidade:

Experimento	Objetivo
`RF_sem_reducao_baseline`	baseline forte sem compressão de features
`RF_com_PCA`	testar compressão não supervisionada com preservação de variância
`RF_com_LDA`	testar redução supervisionada em classificação binária
`DT_sem_reducao_baseline`	comparar ensemble vs. modelo altamente auditável

Todos os experimentos foram executados com validação cruzada estratificada, `GridSearchCV` e rastreamento no MLflow.

3.2 Rastreamento e Reprodutibilidade com MLflow

O `train.py` registra, em cada run:

- hiperparâmetros efetivos do modelo
- métricas de validação cruzada e holdout
- tempo de treino
- artefato serializado do pipeline treinado

Esse desenho garante que a seleção do campeão não dependa de memória operacional ou execução manual dispersa: cada decisão fica associada a um `run_id` recuperável e auditável.

3.3 Seleção e Justificativa do Modelo Campeão

3.3.1 Critérios de Seleção

A escolha do modelo campeão considerou quatro dimensões:

1. **Performance preditiva:** F1-Score no conjunto de teste holdout (métrica primária)
2. **Custo computacional:** tempo de treino e inferência
3. **Interpretabilidade:** capacidade de explicar predições individuais
4. **Viabilidade em produção:** compatibilidade com requisitos regulatórios

3.3.2 Justificativa Técnica

O **Random Forest com PCA (9 componentes)** foi o modelo campeão factual deste ciclo experimental. Ele obteve o maior F1-Score no holdout (`0.4354`) e, por isso, foi a run persistida no MLflow e operacionalizada no Streamlit por meio do `run_id` salvo em `models/champion_run_id.txt`.

Melhor desempenho na métrica primária: Pelo critério definido para seleção do campeão, o `RF + PCA` superou o `RF baseline` por pequena margem absoluta (`0.4354` vs. `0.4308` em F1). Como o objetivo da comparação experimental era maximizar a métrica primária mantendo o pipeline reproduzível, esta foi a escolha adotada para a demonstração funcional.

Trade-off explícito de interpretabilidade: O ganho de performance veio acompanhado de perda de explicabilidade direta. Após o PCA, as importâncias do Random Forest passam a refletir componentes latentes, e não mais as features originais. Em um contexto regulado de crédito, esse é um custo real e precisa ser reconhecido tecnicamente.

Custo computacional ainda aceitável: O experimento com PCA treinou em `24.7 min`, contra `19.4 min` do baseline sem redução. Há aumento de custo, mas ele permanece aceitável para um fluxo acadêmico e para ciclos controlados de re-treino offline.

Decisão de engenharia, não apenas de métrica: Embora o campeão factual deste projeto seja `RF + PCA`, o `RF baseline` segue como forte desafiante por combinar F1 muito próximo, ROC-AUC ligeiramente superior (`0.8572` vs. `0.8555`) e interpretabilidade melhor. Em produção regulada, essa alternativa continuaria plenamente defensável caso a prioridade passasse de performance marginal para auditoria e transparência.

3.3.3 Comparação com o PD1

O modelo campeão do PD1 obteve F1-Score de 0.4328. O PD2 não apenas reproduziu esse valor (RF baseline: 0.4308, diferença de apenas 0.002) como o **superou levemente com RF+PCA: F1=0.4354**. A diferença mínima do baseline é esperada — o split treino/teste do PD2 usa `stratify=y` com `random_state=42`, e o capping adicional de outliers (MonthlyIncome, features de atraso com sentinela 98) muda levemente a distribuição dos dados.

A vantagem do PD2 sobre o PD1 não está no F1-Score, mas na **rastreabilidade**: cada resultado é reprodutível via `run_id` do MLflow, com parâmetros, dados e código completamente documentados.

---

4. Competência 4 — Operacionalização, Inferência e Monitoramento

4.1 Persistência e Versionamento

O modelo campeão é persistido via `mlflow.sklearn.log_model()` como artefato da run MLflow. Esta abordagem garante:

- **Rastreabilidade:** cada modelo no MLflow está associado ao experimento, parâmetros e métricas que o geraram
- **Reprodutibilidade:** o MLflow registra a versão do sklearn e metadados do ambiente de execução
- **Versionamento:** múltiplas versões do modelo coexistem no `mlruns/`, identificadas por `run_id`

4.2 Interface de Inferência (Streamlit)

O aplicativo Streamlit (`app/app.py`) carrega o modelo via `mlflow.sklearn.load_model(model_uri)` e expõe uma interface web com:

- **Sidebar:** formulário com todos os 10 campos de features, com valores padrão razoáveis e texto de ajuda contextualizado
- **Resultado:** classificação "APROVADO (Baixo Risco)" ou "REPROVADO (Alto Risco)" com destaque visual
- **Probabilidades:** gráfico de barras mostrando a distribuição de probabilidade entre as duas classes
- **Explicabilidade:** gráfico de importância de features (disponível apenas para RF sem redução, com mensagem explicativa para modelos com PCA/LDA)

O carregamento usa `@st.cache_resource` para carregar o modelo uma única vez por sessão do servidor, independentemente do número de usuários simultâneos.

4.3 Impacto de Negócio dos Tipos de Erro

Tipo de Erro	Predição	Realidade	Impacto
Falso Negativo	Adimplente	Inadimplente	Aprovação de crédito para quem vai inadimplir → perda financeira direta
Falso Positivo	Inadimplente	Adimplente	Recusa de crédito a bom pagador → perda de receita, risco reputacional

Assimetria de custo: Em crédito, o Falso Negativo tipicamente tem custo maior que o Falso Positivo. O F1-Score trata os dois igualmente — em produção, seria apropriado ponderar pelo custo relativo de cada erro e ajustar o threshold de decisão da probabilidade (padrão: 0.5).

4.4 Métricas Técnicas Monitoradas

Métrica	Propósito	Alerta
F1-Score (janela 30 dias)	Degradação do modelo	Queda > 5% vs. baseline
ROC-AUC	Capacidade discriminativa	Queda > 0.03
Taxa de inadimplência prevista	Sanidade do modelo	Desvio > 20% da média histórica
Latência de inferência	Performance operacional	P99 > 500ms

4.5 Detecção de Data Drift

Data drift ocorre quando a distribuição das features de entrada em produção diverge da distribuição do conjunto de treino, degradando a performance sem que o modelo "saiba" disso.

Estratégias propostas:

PSI (Population Stability Index): Para cada feature contínua, calcular o PSI entre a distribuição de treino e janelas de 30 dias de produção. - PSI < 0.1: distribuições estáveis (sem ação) - 0.1 ≤ PSI < 0.25: mudança moderada (investigar) - PSI ≥ 0.25: mudança severa (re-treino prioritário)

Teste de Kolmogorov-Smirnov: Para features numéricas, KS-test com p-value < 0.01 indica drift significativo.

Monitoramento da distribuição do target: Se a taxa de inadimplência observada (após 90 dias de carência) divergir significativamente da taxa prevista, o modelo está descalibrado.

Gatilhos de re-treino: 1. PSI ≥ 0.25 em qualquer feature crítica (`RevolvingUtilizationOfUnsecuredLines` , `DebtRatio`) 2. F1-Score em produção < 0.35 por duas semanas consecutivas 3. Mudanças macroeconômicas significativas (ex: mudança da taxa Selic, crises econômicas)

4.6 Estratégia de Re-treino Contínuo

O MLflow facilita o re-treino controlado: cada ciclo de re-treino cria novos `run_id` s que permitem comparar a versão nova com a versão em produção antes de fazer o swap. O fluxo recomendado:

```
Dados novos → train.py → MLflow (nova run) →
Comparação de métricas (novo vs. atual) →
Aprovação → Atualizar champion_run_id.txt →
Reiniciar Streamlit
```

---

5. Conclusões

5.1 O que Este Projeto Demonstrou

1. **A transição do notebook para sistema modular é uma mudança de mindset**, não apenas de formato. Módulos separados forçam a explicitação de interfaces, contratos e responsabilidades que o notebook escondia.
2. **Redução de dimensionalidade exige análise de trade-off, não adesão automática.** Com 10 features e 150.000 registros, o LDA degradou fortemente o desempenho e o PCA trouxe ganho marginal de F1 ao custo de interpretabilidade. A evidência experimental mostra que performance isolada e auditabilidade podem apontar para escolhas diferentes.
3. **Rastreamento é infraestrutura, não burocracia.** O MLflow elimina a pergunta "qual versão do modelo está em produção?" — a resposta está sempre no `run_id` registrado, com todos os parâmetros e métricas associados.
4. **Deploy não é o fim do ciclo de ML, é o começo.** O sistema de monitoramento proposto (PSI, KS-test, F1 em produção) fecha o loop entre dados, modelo e negócio — tornando o sistema verdadeiramente operacional.

5.2 Próximos Passos Sugeridos

- Implementar pipeline de monitoramento automatizado com alertas (ex: via Airflow ou scheduled job)
- Explorar modelos com maior interpretabilidade nativa (ex: LGBM com explicações SHAP)
- Adicionar autenticação e logging de predições ao Streamlit para auditoria regulatória
- Containerizar o sistema (Docker) para deploy em ambiente de produção real

---

Referências

- LGPD — Lei Geral de Proteção de Dados (Lei 13.709/2018), Artigo 20
- Banco Central do Brasil — Resolução CMN nº 4.557/2017 (gestão de riscos)
- Kaggle: Give Me Some Credit Dataset — https://www.kaggle.com/datasets/brycecf/give-me-some-credit-dataset
- MLflow Documentation — https://mlflow.org/docs/latest/
- Scikit-learn User Guide: Pipelines and composite estimators
- Grus, Joel. *Data Science from Scratch*. O'Reilly, 2019.
- Kleppmann, Martin. *Designing Data-Intensive Applications*. O'Reilly, 2017.